

FitBuddy-AI: Fitness Plan Generator

Project Overview: This complete project bundle contains the full source code for an AI-powered fitness and nutrition plan generator utilizing Google Gemini modules (Gemini 1.5 Flash / Pro via the Google Gen AI SDK). The architecture consists of a Node.js Express backend and a clean, responsive frontend dashboard.

1. Project Configuration (package.json)

```
{
  "name": "fitbuddy-ai-fitness-generator",
  "version": "1.0.0",
  "description": "AI Fitness & Nutrition Plan Generator using Gemini Modules",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "@google/generative-ai": "^0.21.0",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2"
  }
}
```

2. Backend Implementation (server.js)

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const { GoogleGenAI } = require('@google/generative-ai');

dotenv.config();

const app = express();
app.use(cors());
app.use(express.json());
app.use(express.static('public'));

// Initialize Gemini API Client
// Make sure GOOGLE_API_KEY is set in your .env file
const ai = new GoogleGenAI({ apiKey: process.env.GOOGLE_API_KEY });

app.post('/api/generate-plan', async (req, res) => {
  try {
    const { name, age, weight, goal, level, location, dietary } = req.body;

    // Utilize Gemini 1.5 Flash for high-speed structural generation
    const model = ai.getGenerativeModel({ model: 'gemini-1.5-flash' });

    const prompt = `
    You are an expert AI fitness coach and clinical nutritionist.
    Create a highly customized 7-day fitness and diet blueprint for the following user:
    - Name: ${name}
    - Age: ${age}
    - Weight: ${weight} kg
    - Fitness Goal: ${goal}
    - Experience Level: ${level}
    - Preferred Workout Location: ${location}
    - Dietary Restrictions: ${dietary}

    Provide your response in clear, structured sections:
    1. WEEKLY WORKOUT PLAN (Specify exercises, sets, reps, and resting guidelines based
    on their location and level)
    2. WEEKLY DIET PLAN (Provide daily caloric layout & meal macro structures matching
    their dietary preferences)
    3. COACH'S MOTIVATION & ESSENTIAL TIPS
    `;

    const result = await model.generateContent(prompt);
    const responseText = result.response.text();

    res.json({ success: true, plan: responseText });
  } catch (error) {
    console.error('Error generating blueprint:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`FitBuddy Backend running smoothly on port ${PORT}`);
});
```

```
});
```

3. Frontend Layout (public/index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>FitBuddy-AI Plan Generator</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <header>
      <h1>FitBuddy-AI</h1>
      <p>Hyper-Personalized Fitness & Nutrition Planner via Gemini Modules</p>
    </header>

    <main class="grid">
      <section class="form-card">
        <h2>Your Profile Metrics</h2>
        <form id="fitnessForm">
          <div class="form-group">
            <label for="name">Full Name</label>
            <input type="text" id="name" required placeholder="Alex Morgan">
          </div>
          <div class="form-row">
            <div class="form-group">
              <label for="age">Age</label>
              <input type="number" id="age" required placeholder="28">
            </div>
            <div class="form-group">
              <label for="weight">Weight (kg)</label>
              <input type="number" id="weight" required placeholder="75">
            </div>
          </div>
          <div class="form-group">
            <label for="goal">Fitness Goal</label>
            <select id="goal">
              <option value="Fat Loss & Toning">Fat Loss & Toning</option>
              <option value="Muscle Building & Hypertrophy">Muscle Building
& Hypertrophy</option>
              <option value="Endurance & Cardiorespiratory">Endurance & Car
diorespiratory</option>
            </select>
          </div>
          <div class="form-group">
            <label for="level">Fitness Experience Level</label>
            <select id="level">
              <option value="Beginner">Beginner</option>
              <option value="Intermediate">Intermediate</option>
              <option value="Advanced">Advanced</option>
            </select>
          </div>
          <div class="form-group">
            <label for="location">Preferred Environment</label>
            <select id="location">
              <option value="Gym (Full Equipment)">Gym (Full Equipment)</op
```

```

tion>
        <option value="Home (Bodyweight/Dumbbells)">Home (Bodyweight/
Dumbbells)</option>
    </select>
</div>
<div class="form-group">
    <label for="dietary">Dietary Preferences</label>
    <input type="text" id="dietary" placeholder="e.g. Vegetarian, Hig
h-Protein, Keto">
</div>
<button type="submit" id="submitBtn">Generate My AI Blueprint</button
>
    </form>
</section>

<section class="result-card">
    <h2>Your Custom AI Blueprint</h2>
    <div id="loader" class="hidden">Generating your optimized regime...</div>
    <div id="planOutput" class="markdown-body">
        Your generated schedule will display here. Fill out the metrics profi
le to begin.
    </div>
</section>
</main>
</div>
<script src="script.js"></script>
</body>
</html>

```

4. Frontend Styling (public/style.css)

```
* { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Segoe UI', system-ui, sans-serif; }
body { background: #f0f2f5; color: #1f2937; padding: 20px; }
.container { max-width: 1200px; margin: 0 auto; }
header { text-align: center; margin-bottom: 30px; }
header h1 { color: #1e3a8a; font-size: 2.5rem; }
header p { color: #6b7280; font-size: 1.1rem; }
.grid { display: grid; grid-template-columns: 1fr 1fr; gap: 30px; }
@media (max-width: 768px) { .grid { grid-template-columns: 1fr; } }
.form-card, .result-card { background: white; padding: 25px; border-radius: 12px; box-shadow: 0 4px 6px rgba(0,0,0,0.05); }
h2 { margin-bottom: 20px; color: #2563eb; font-size: 1.4rem; border-bottom: 2px solid #eff6ff; padding-bottom: 8px; }
.form-group { margin-bottom: 15px; }
.form-row { display: grid; grid-template-columns: 1fr 1fr; gap: 15px; }
label { display: block; margin-bottom: 5px; font-weight: 600; font-size: 0.9rem; color: #4b5563; }
input, select { width: 100%; padding: 10px; border: 1px solid #d1d5db; border-radius: 6px; font-size: 1rem; }
button { width: 100%; padding: 12px; background: #2563eb; color: white; border: none; border-radius: 6px; font-size: 1.1rem; font-weight: 600; cursor: pointer; transition: background-color 0.2s; }
button:hover { background: #1d4ed8; }
.hidden { display: none; }
.loader { font-weight: bold; color: #2563eb; text-align: center; padding: 20px; }
.markdown-body { white-space: pre-wrap; line-height: 1.6; font-size: 0.95rem; color: #374151; }
```

5. Frontend Interactivity Logic (public/script.js)

```
document.getElementById('fitnessForm').addEventListener('submit', async (e) => {
  e.preventDefault();

  const submitBtn = document.getElementById('submitBtn');
  const loader = document.getElementById('loader');
  const planOutput = document.getElementById('planOutput');

  const formData = {
    name: document.getElementById('name').value,
    age: document.getElementById('age').value,
    weight: document.getElementById('weight').value,
    goal: document.getElementById('goal').value,
    level: document.getElementById('level').value,
    location: document.getElementById('location').value,
    dietary: document.getElementById('dietary').value || 'None'
  };

  submitBtn.disabled = true;
  loader.classList.remove('hidden');
  planOutput.innerHTML = '';

  try {
    const response = await fetch('/api/generate-plan', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(formData)
    });

    const data = await response.json();
    if (data.success) {
      planOutput.innerHTML = data.plan;
    } else {
      planOutput.innerHTML = `

Error: ${data.error}</p>`;
    }
  } catch (err) {
    planOutput.innerHTML = `

Network error occurred. Please try again.</p>`;
  } finally {
    submitBtn.disabled = false;
    loader.classList.add('hidden');
  }
});


```

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.